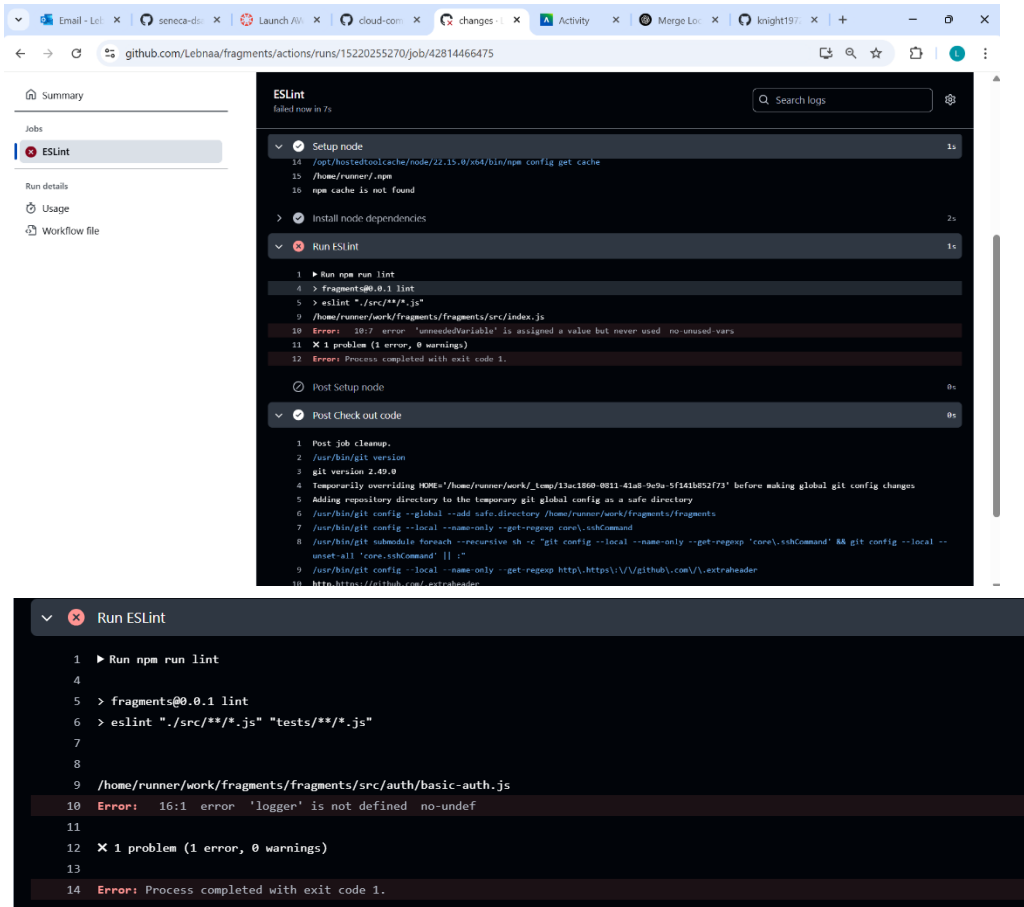


Screenshot of your CI workflow failing ESLint



Screenshot of your CI workflow failing one or more Unit Tests



```
59 FAIL tests/unit/get.test.js
60   • Test suite failed to run
61     env contains configuration for both AWS Cognito and HTTP Basic Auth. Only one is allowed.
62   10 |   process.env.HTPASSWD_FILE
63   11 | } {
64   > 12 |   throw new Error(
65       |         ^
66   13 |   'env contains configuration for both AWS Cognito and HTTP Basic Auth. Only one is allowed.'
67   14 |   );
68   15 | }
69   at Object.<anonymous> (src/auth/index.js:12:9)
70   at Object.require (src/routes/index.js:8:26)
71   at Object.require (src/app.js:46:14)
72   at Object.require (tests/unit/get.test.js:5:13)
```

Screenshot of your CI workflow passing ESLint

← → ↻ github.com/Lebnaa/fragments/actions/runs/15222483478/job/42820158702 🔍 ⭐ 📄 ⌵

✔ Update envjest #11 Re-run all jobs ⋮

Summary

Jobs

- ✔ ESLint
- ✔ Unit Tests

Run details

- 🕒 Usage
- 📄 Workflow file

ESLint

succeeded 4 minutes ago in 10s

🔍 Search logs 🔁 ⚙️

> Set up job 1s

> Check out code 1s

> Setup node 2s

▼ Install node dependencies 3s

1 ▶ Run npm ci

4 npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerful.

5 npm warn deprecated glob@7.2.3: Glob versions prior to v9 are no longer supported

6

7 added 481 packages, and audited 482 packages in 3s

8

9 78 packages are looking for funding

10 run 'npm fund' for details

11

12 found 0 vulnerabilities

▼ Run ESLint 6s

1 ▶ Run npm run lint

4

5 > fragments@0.0.1 lint

6 > eslint -c ./.eslintrc.js "tests/**/*.js"

7

> Post Setup node 0s

> Post Check out code 1s

> Complete job 0s

Screenshot of your CI workflow passing all Unit Tests

The screenshot shows a CI workflow summary for a project named 'Update envjest'. The 'Unit Tests' job is highlighted, showing a list of steps: 'Set up job', 'Check out code', 'Setup node', and 'Install node dependencies and run tests'. The 'Install node dependencies and run tests' step is expanded, showing a terminal log. The log indicates that the tests passed successfully, with a summary showing 11 passed, 0 failed, and 0 skipped tests. The total time for the tests was 0.003 s.

Screenshot of your test coverage report, showing the app.js 404 handler being covered. Open the file `coverage/lcov-report/src/app.js.html` in your browser to show the lines of code being hit (green) by the tests, and those being missed (red).

The screenshot shows a test coverage report for the file `app.js`. The report is split into two columns: the left column shows the source code with line numbers, and the right column shows the corresponding test code. The source code is color-coded to indicate coverage: green for lines that were hit by the tests and red for lines that were not. The test code is also color-coded, with green for lines that passed and red for lines that failed. The report shows that the 404 handler in `app.js` was covered by the tests.